

ALU Blockchain Asset Protection System

Project Report — Blockchain Development Assignment

African Leadership University | 2024/2025 Academic Year

Abstract

This project implements a two-contract Ethereum blockchain system to protect and tokenise the African Leadership University (ALU) official logo. ALUAssetRegistry (ERC-721) registers the logo as a non-fungible token and stores a SHA-256 cryptographic fingerprint permanently on-chain, enabling anyone to verify whether a logo file is authentic. ALULogoToken (ERC-20) issues one million divisible ALUT tokens that represent fractional ownership shares of the logo among university stakeholders. Both contracts were deployed and tested on the Ethereum Sepolia testnet.

Introduction

The ALU official logo is a core institutional identity asset. Counterfeit or modified versions can appear in fraudulent communications or unofficial merchandise, damaging the university's reputation. Traditional protection methods each have critical limitations: copyright registration depends on slow legal enforcement; a database record is stored on a server controlled by one party and can be altered or deleted; and watermarks can be removed by basic image editing.

Blockchain technology offers protection that none of these methods can match. A blockchain is a distributed ledger maintained simultaneously across thousands of independent computers. Once data is written to a confirmed block, altering it would require re-computing every subsequent block across more than half of the entire network — a task that is economically impractical. A SHA-256 fingerprint of the ALU logo, once recorded on-chain, is permanent, public, and verifiable by anyone without requiring trust in any central authority.

The project uses Ethereum because it supports smart contracts: self-executing programs that enforce their own terms without any intermediary. The OpenZeppelin library provides industry-audited ERC-721 and ERC-20 implementations. Development uses Hardhat, a professional Ethereum framework providing a local blockchain, automated testing with Mocha and Chai, and deployment scripting.

Question 1: Cryptocurrency Exchange Exploration (Coinbase)

Step 1 — Account Creation and Identity Verification

Registering on Coinbase requires an email address, a password, and identity verification (Know Your Customer, or KYC). The platform asks for a government-issued photo ID, a real-time selfie for liveness detection, and a home address. Depending on jurisdiction, proof of address such as a utility bill may also be required. KYC is mandatory because cryptocurrency exchanges are classified as financial institutions under Anti-Money Laundering (AML) law. In Rwanda, the

National Bank of Rwanda enforces equivalent requirements. Without identity verification, exchanges could be used anonymously to launder proceeds from criminal activity.

Step 2 — Platform Exploration

After logging in, the dashboard shows portfolio balances, price charts, and a quick-buy panel. Three cryptocurrencies available on the platform:

- Bitcoin (BTC): The original cryptocurrency, used as a store of value and medium of exchange. Its distinctive feature is a hard-coded supply cap of 21 million coins — scarcity guaranteed by code, not by any authority.
- Ethereum (ETH): A programmable blockchain that hosts smart contracts and decentralised applications. Unlike Bitcoin, its purpose is computation, not just value transfer. This is the network used in this project.
- Chainlink (LINK): A decentralised oracle network that feeds real-world data (prices, weather) to smart contracts on the blockchain. Unlike the other two, it is specialised infrastructure, not a general-purpose currency.

Step 3 — Adding a Payment Method

To add a bank account, Coinbase requires the account number and routing number and verifies ownership via micro-deposits. A debit card requires the card number, expiry date, and CVV. Bank transfers cost approximately 1.49% in fees but take 3–5 business days. Debit cards are instant but charge approximately 3.99%. Bank transfer is cheaper; debit card is faster.

Step 4 — Simulating a \$100 ETH Purchase

A market order executes immediately at the current market price. A limit order only fills when ETH reaches a price you specify in advance. For a straightforward \$100 purchase, a market order was chosen to guarantee instant execution. At a 3.99% debit card fee, \$3.99 goes to Coinbase, leaving \$96.01 to buy ETH. At approximately \$3,500 per ETH, this yields roughly 0.0274 ETH.

Step 5 — Transaction Records

The transaction history shows: date, cryptocurrency, fiat amount spent, crypto amount received, fee, and the transaction hash. A transaction hash (also called txid or txhash) is a unique 64-character hexadecimal string that permanently identifies a specific transaction on the blockchain. It is important because it functions as a public, immutable receipt — anyone can enter it into a block explorer such as Etherscan.io and independently verify exactly what was transferred, to which wallet address, at which block height, and at what time, without relying on Coinbase's internal records.

Step 6 — Security Features

- Two-Factor Authentication (2FA): Requires a six-digit rotating code from an authenticator app in addition to the password. Even if the password is stolen, the account cannot be accessed without the physical phone.
- Withdrawal Address Allowlisting: Only pre-approved wallet addresses can receive withdrawals. Any attempt to send funds to a new address triggers a 48-hour delay and an email confirmation. This stops attackers from draining the account even after gaining full access.

Question 2: Blockchain Setup (Part A — Hardhat)

Hardhat is a professional Ethereum development framework. Running `npx hardhat node` starts a local Ethereum blockchain entirely on the developer's machine, creates 20 test wallet accounts each pre-loaded with 10,000 test ETH, and mines blocks automatically when transactions arrive.

Developers use a local blockchain instead of the real Ethereum mainnet for three reasons: mainnet deployment costs real ETH in gas fees; the local network resets to a clean state on restart, making reproducible testing straightforward; and local blocks confirm near-instantly, whereas the mainnet takes 12–15 seconds per block.

A wallet address is a 40-character hexadecimal string (prefixed 0x) that uniquely identifies an Ethereum account. It is derived by hashing the account's public key with Keccak-256. It serves as both an identity (who owns tokens or NFTs) and a destination (where to send transactions).

Sample output from `npx hardhat node` (local test network — not real funds):

```
$ npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/
Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
```

[Screenshot of your actual `npx hardhat node` terminal output here]

Question 2: Smart Contracts (Part B — ALUAssetRegistry)

What Is a Smart Contract?

A smart contract is a program stored on a blockchain and executed automatically when its conditions are met. Unlike a programme on a company server — which can be modified, shut down, or misreported by whoever controls it — a smart contract on Ethereum runs identically on every node in the network. Its code is public, its execution is guaranteed, and its results are permanently on-chain. It enforces its own terms without any trusted intermediary.

What Is an NFT and Why ERC-721?

An NFT (Non-Fungible Token) is a unique, non-interchangeable digital token. Each NFT has its own identifier that cannot be replicated. ERC-721 is the Ethereum standard that defines the interface for NFTs. The ALU logo is a unique institutional asset — there is exactly one official version — so ERC-721 is correct. A fungible ERC-20 token would be wrong because every unit is identical, with no way to represent a single, specific logo record.

What Is SHA-256 and Why Is One Pixel Change Fatal?

SHA-256 is a cryptographic hash function that produces a fixed 64-character hexadecimal output from any input. Any change — even one pixel — propagates unpredictably through 64 rounds of bitwise operations, producing a completely different output. The SHA-256 hash of the ALU logo used in this project:

```
sha256sum alu-logo.png
b2a1ba00a20638d7d1960886a9d7ca45a374865ddc1d69f425c4304b97ff421d  alu-logo.png
```

[Screenshot of your sha256sum command output here]

Why Blockchain Data Cannot Be Changed

Every Ethereum block contains the hash of the previous block. Changing any historical record would change that block's hash, which invalidates every subsequent block's hash. An attacker would need to re-mine every block since the tampered one faster than the honest network — requiring over 50% of the entire network's computing power, which is economically infeasible.

Contract Functions

registerAsset(assetName, fileType, contentHash)

Checks that the SHA-256 hash has not been registered before — reverts with a clear error message if it has. If new: increments a counter, mints an ERC-721 NFT to the caller via `_safeMint()`, saves the `AssetMetadata` struct (name, file type, hash, caller address, Unix timestamp) to storage, marks the hash as used to prevent duplicates, emits an `AssetRegistered` event, and returns the new token ID.

verifyLogoIntegrity(tokenId, contentHash)

A gas-free view function. Compares the caller's provided hash against the stored hash for the given token ID. Returns (true, 'Logo is authentic.') if they match, or (false, 'Warning: logo does not match.') otherwise. Anyone can use this to verify whether a logo file is the official registered version.

getAsset(tokenId)

A gas-free view function. Returns the complete `AssetMetadata` struct for the given token: asset name, file type, SHA-256 hash, registering wallet address, and Unix registration timestamp. Reverts if the token does not exist.

Testnet Deployment (Sepolia)

After verifying all tests locally, both contracts were deployed to the Ethereum Sepolia public testnet:

1. Created a `.env` file with a Sepolia wallet private key and Alchemy RPC URL.
2. Obtained free Sepolia ETH from <https://sepoliafaucet.com> to cover gas fees.
3. Ran: `npx hardhat run scripts/deploy.js --network sepolia`
4. Verified both contracts on Sepolia Etherscan using the printed addresses.

ALUAssetRegistry on Sepolia: [Insert address + Etherscan link]

ALULogoToken on Sepolia: [Insert address + Etherscan link]

[Screenshot of Sepolia Etherscan showing both deployed contracts here]

Question 2: Tokenisation (Part C — ALULogoToken)

What Is Tokenisation?

Tokenisation is the process of representing ownership of an asset as blockchain tokens that can be divided, held, and transferred. For the ALU logo, 1,000,000 ALUT tokens are minted at deployment. ALU administration might retain 700,000 tokens (70%), distribute 100,000 (10%) to the Faculty Senate, 50,000 (5%) to Student Government, and reserve 150,000 (15%) for alumni and future partners. Each holder's stake is permanently recorded on-chain and publicly verifiable.

ERC-721 vs ERC-20

Property	ERC-721 — ALUAssetRegistry	ERC-20 — ALULogoToken
Fungibility	Non-fungible (each token unique)	Fungible (all tokens identical)
Purpose	Prove unique ownership of one logo	Share ownership across stakeholders
Divisible?	No — one NFT = one logo record	Yes — 1,000,000 equal shares

A faculty member holding 100,000 ALUT out of 1,000,000 holds a 10% ownership stake. In practice, this means they have a verifiable, on-chain claim to proportional say in any licensing decision for the logo and are entitled to a 10% share of any licensing revenue if the logo is commercially licensed. Their stake cannot be altered without their consent or a formal governance vote.

Token Functions

distributeShares(recipient, amount)

Restricted to the contract owner via the `onlyOwner` modifier from OpenZeppelin. Validates that `amount > 0` and recipient is not the zero address, then calls ERC-20's `transfer()` to move ALUT tokens from the owner's wallet to the recipient — the mechanism through which ALU distributes ownership stakes.

ownershipPercentage(wallet)

A gas-free view function. Returns $(\text{balance} \times 100) / \text{totalSupply}()$ as a whole integer. Example: 500,000 ALUT → 50; 100,000 ALUT → 10. Values below 1% are truncated to 0 because the return type is `uint256`.

Test Results

All eight automated tests pass with 0 failures (`npx hardhat test`):

```
$ npx hardhat test
  ALUAssetRegistry (ERC-721)
  ✓ Test 1 – registers ALU logo successfully and emits AssetRegistered with tokenId 1
  ✓ Test 2 – rejects duplicate registration of the same content hash
```

```
✓ Test 3 – verifyLogoIntegrity() returns true when the correct hash is supplied
✓ Test 4 – verifyLogoIntegrity() returns false when an incorrect hash is supplied
✓ Test 5 – getAsset() returns the correct asset name and file type

ALULogoToken (ERC-20)
✓ Test 6 – mints the full supply of 1,000,000 ALUT to the logo owner on deployment
✓ Test 7 – distributeShares() transfers the specified token amount to a recipient
✓ Test 8 – ownershipPercentage() returns the correct whole-number percentage

8 passing (978ms)
```

[Screenshot of this terminal output must be inserted here for submission]

Conclusion

This project demonstrates how blockchain technology provides permanent, tamper-proof protection for an institutional digital asset. The two-contract system — ALUAssetRegistry (ERC-721) for logo registration and ALULogoToken (ERC-20) for fractional ownership — ensures that the ALU logo has a verifiable on-chain provenance record and that stakeholder ownership is transparently distributed and auditable by anyone.

Key lessons: the distinction between ERC-721 (unique assets) and ERC-20 (divisible shares) is fundamental; OpenZeppelin provides secure, audited building blocks that reduce smart contract risk significantly; and SHA-256 hashing is a practical, computationally cheap mechanism for creating a verifiable file fingerprint.

Given more time, three improvements would strengthen the system: (1) a multi-signature ownership model requiring multiple parties to approve any governance action; (2) IPFS integration to store the logo file itself (not just its hash) on decentralised storage; and (3) a governance contract allowing ALUT holders to vote on licensing decisions proportional to their stake.

GitHub Repository & Video Demo

GitHub repository (all source code, public):

<https://github.com/Phinatech/alu-blockchain-logo-protection>

[Replace with your actual public repository URL before submitting]

Demo video link:

[Insert your video link here — YouTube, Loom, or Google Drive — showing contracts deployed and tests passing]